

RSE-Supported Scientific Software Standardization

Spencer Smith, Christian Brodbeck, Zhangwenchi Li

Computing and Software Department
Faculty of Engineering
McMaster University
Hamilton, Ontario, Canada

IRSC26, Sept 8, 2026

Smith et al.

Abstract

Background

Steps

Standardize?
Backward?
Learn
Curr. Architec.?
Abstraction
Implement
Verification
Migration

Guidelines

Concluding Remarks

References

- Technical debt → Standardization → Interface
- Steps
 - ① Should you standardize?
 - ② Backward compatibility?
 - ③ Learn necessary domain knowledge
 - ④ Document current design architecture
 - ⑤ Abstraction to isolate interface
 - ⑥ Implementation
 - ⑦ Verification
 - ⑧ Migration plan
- Guidelines
 - Question-driven learning
 - Information hiding
 - Task decomposition
 - Multi-layered testing
 - Deprecation warnings
- Case study: Eelbrain-BIDS

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

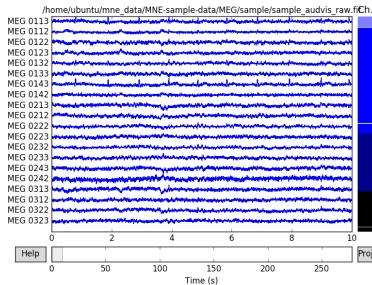
Migration

Guidelines

Concluding
Remarks

References

- Magnetoencephalography (MEG)
- Electroencephalography (EEG)
- Stimulus, then record a multi-channel time series



MEG device and the recorded multi-channel data

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

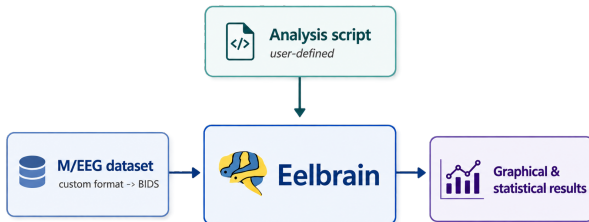
Migration

Guidelines

Concluding
Remarks

References

- Open-source Python library for M/EEG data analysis¹
- BIDS (Brain Imaging Data Structure) is a community standard for organizing and sharing brain data
- The Eelbrain-BIDS project upgraded the input dataset format of Eelbrain from a custom format to BIDS



¹<https://github.com/Eelbrain/Eelbrain>

Smith et al.

Abstract

Background

Steps

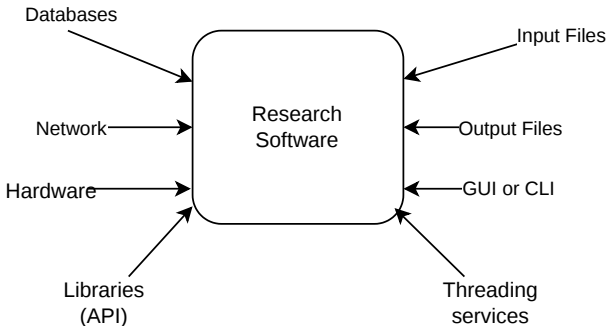
Standardize?
Backward?
Learn
Curr. Architect.?
Abstraction
Implement
Verification
Migration

Guidelines

Concluding
Remarks

References

Technical debt² → Standardization → Interface standards



²Term first used by [Cunningham \(1992\)](#)

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

Pros

Cons

Interoperability³⁴

Reproducibility³

Reusability³

Productivity

Communication

Cost of paying technical debt

Have to update to avoid obsolescence

Lose “benefits” of closed data formats

Standard may not be a perfect fit

Standards may not always be best

- For Eelbrain moving to BIDS
 - Data sharing since open data is often in BIDS format
 - Community has converged on BIDS
 - Start with six BIDS entities, then add more as needed

³Basu et al. (2019)

⁴David and Greenstein (1990)

Smith et al.

Make Backward Compatible:

Reasons to keep	Reasons to break
Existing code still works	Time+effort to keep
Facilitates transition	Future maintenance easier
Reproducibility	Still have tech. debt.
Protects downstream soft.	Removes constraints
User trust	Reduces testing burden

- Depends on current architecture
- For Eelbrain-BIDS, backward compatibility was dropped – BIDS makes different assumptions, too much repetition of code, not enough demand to justify

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architect.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding

Remarks

References

- Don't need to understand everything, just enough to understand the interface
- Ask domain expert questions⁵
 - What keyword should I search for?
 - What is the problem that the software is trying to solve?
 - What is the context in which the software is used?
 - What are the typical use cases for the software?
 - **What is the approximate scale of the problem.**
 - What should the user provide as **inputs**?
 - What **output** does the user expect?
 - What are trends in the output as the input varies?
- For a well-documented project, an AI chat-bot could substitute for a domain expert

⁵Du and Smith (2025)

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architect.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

- Need current architecture for planning
- If the interface is well-defined, change can be isolated
- If not, the first step is to refactor
- Use information hiding principle⁶

⁶Parnas (1972)

Smith et al.

Solve $\frac{dT_W}{dt} = \frac{1}{\tau_W}(T_C - T_W(t))$, $T_W(0) = T_{init}$

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

```

1  t = [0.0]
2  T_W = [T_init]
3
4  while t[-1] < t_end:
5      T = T_W[-1]
6
7      # Euler update
8      T_new = T + dt * (T_C - T) / tau_W
9      t_new = t[-1] + dt
10
11     t.append(t_new)
12
13     T_W.append(T_new)

```

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

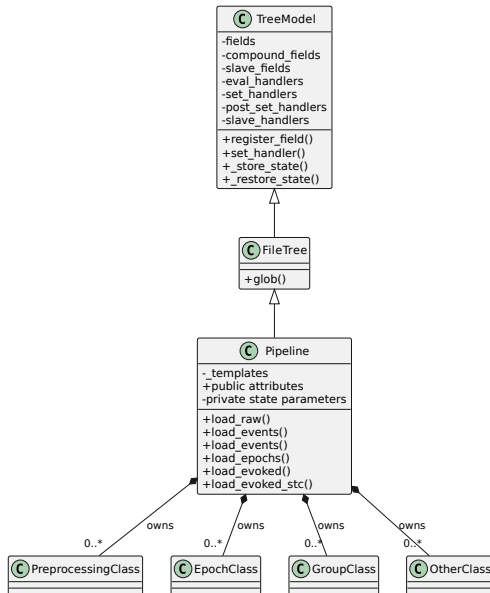
Concluding
Remarks

References

```

1  def dT_W_dt(t, T_W, T_C, tau_W):
2      return (T_C - T_W) / tau_W
3
4  # Bind the parameters of our particular ODE
5  def derivative(t, T_W):
6      return dT_W_dt(t, T_W, T_C, tau_W)
7
8  t_eval = np.arange(0, t_end + dt, dt)
9
10 solution = solve_ivp(
11     derivative,
12     (0, t_end),
13     [T_init],
14     t_eval=t_eval,
15     method="RK45"
16 )

```



Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

- Eelbrain-BIDS
 - Changes mostly to Pipeline
 - Missed path templating
- Degree of abstraction depends on likelihood of change
- Wrapper (Adapter) design pattern in trivial cases
- Add strategy pattern for backward compatibility
- Requires a mapping between the old and new interface
- Not an option for Eelbrain-BIDS

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

- Which library?
 - Performance, robustness, documentation quality, learning curve, community support, etc.
 - Flexibility vs. simplicity
 - Community size
- **Task decomposition (helps non-experts)**
- Assessing change impact, search code
- Code reading using debug mode
- Incremental iterative development
- Eelbrain-BIDS
 - Upgrade parent classes first
 - Domain knowledge in steps: preprocessing, epochs, source localization, etc.
 - Stick with `mne.read_raw_fiff` over `mne-bids`

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architect.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

- Multi-layered testing (unit, integ., system)
- Unit tests, integration tests, system tests
- Reuse existing tests
- Continuous integration (CI)
- **Update documentation with each pull request**

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

- Version number as clearest signal of breaking changes⁷
- Release notes
- Deprecation warnings
- Evolution models
 - Old and new interfaces coexist for a while, then old interface is deprecated
 - Old interface is deprecated immediately (two in production)⁸
- Eelbrain-BIDS uses two in production

⁷ [Preston-Werner \(2023\)](#)

⁸ [Lübke et al. \(2019\)](#)

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

- Case study: Eelbrain-BIDS
- Steps
 - 1 Should you standardize?
 - 2 Backward compatibility?
 - 3 Learn necessary domain knowledge
 - 4 Document current design architecture
 - 5 Abstraction to isolate interface
 - 6 Implementation
 - 7 Verification
 - 8 Migration plan
- Guidelines/techniques
 - Question-driven learning
 - Information hiding
 - Task decomposition
 - Multi-layered testing
 - Deprecation warnings

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding
Remarks

References

Amrita Basu, Denise Warzel, Aras Eftekhari, Justin Kirby, John Freymann, Janice Knable, Ashish Sharma, and Paula Jacobs. Call for data standardization: Lessons learned and recommendations in an imaging study. *JCO clinical cancer informatics*, (3):1–11, Dec 2019. doi: <https://doi.org/10.1200/cci.19.00056>.

Ward Cunningham. The wycash portfolio management system. In *Addendum to the Proceedings on Object-Oriented Programming Systems, Languages, and Applications (Addendum)*, OOPSLA '92, pages 29–30, New York, NY, USA, 1992. Association for Computing Machinery. ISBN 0897916107. doi: [10.1145/157709.157715](https://doi.org/10.1145/157709.157715). URL <https://doi.org/10.1145/157709.157715>.

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding

Remarks

References

Paul David and Shane Greenstein. The economics of compatibility standards: An introduction to recent research. *Economics of Innovation and New Technology*, 1:3–41, 01 1990. doi: 10.1080/104385990000000002.

Phil Du and W. Spencer Smith. A practical framework for small teams to develop sustainable research software. In *Improving Scientific Software Conference (ISSC)*, pages 1–9, June 2025. doi: <https://zenodo.org/records/17023784>. URL <https://ucar-sea.github.io/SEA-ISS-2025-A-Practical-Framework-for-Small-Teams-to-DRAFT-README.html>.

Daniel Lübke, Olaf Zimmermann, Cesare Pautasso, Uwe Zdun, and Mirko Stocker. Interface evolution patterns. *Proceedings of the 24th European Conference on Pattern Languages of Programs - EuroPLop '19*, 2019. doi: <https://doi.org/10.1145/3361149.3361164>.

Smith et al.

Abstract

Background

Steps

Standardize?

Backward?

Learn

Curr. Architec.?

Abstraction

Implement

Verification

Migration

Guidelines

Concluding

Remarks

References

D. L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15 (12):1053–1058, Dec 1972. doi:

<https://doi.org/10.1145/361598.361623>. URL

<https://dl.acm.org/doi/10.1145/361598.361623>.

Tom Preston-Werner. Semantic versioning 2.0.0, 2023.

URL <https://semver.org/>.